COM 333:DESIGN AND ANALYSIS OF ALGORITHMS

Chapter 4: Algorithm Design Techniques

Computer Science & Informatics department

Algorithm classification

- Algorithms that use a similar problem-solving approach can be grouped together
- We'll talk about a classification scheme for algorithms
- This classification scheme is neither exhaustive nor disjoint
- The purpose is not to be able to classify an algorithm as one type or another, but to highlight the various ways in which a problem can be attacked

Algorithm Design Techniques

- Brute Force
- Greedy Algorithms
- Divide and Conquer
- Dynamic Programming
- Transform and Conquer
- Backtracking
- Genetic Algorithms

1. Brute force algorithms

- A brute force algorithm simply tries *all* possibilities until a satisfactory solution is found
  - Such an algorithm can be:
    - **Optimizing**: Find the *best* solution. This may require finding all solutions, or if a value for the best solution is known, it may stop when any best solution is found
      - Example: Finding the best path for a traveling salesman
    - **Satisficing**: Stop as soon as a solution is found that is *good enough*
      - Example: Finding a traveling salesman path that is within 10% of optimal

Improving Brute Force algorithms

- Often, brute force algorithms require exponential time
- Various *heuristics* and *optimizations* can be used
  - Heuristic: A “rule of thumb” that helps you decide which possibilities to look at first
  - Optimization: In this case, a way to eliminate certain possibilities without fully exploring them

Brute Force contd.

- **Based on the problem's statement and definitions of the concepts involved.**
- **Examples:**
  - Sequential search
  - Exhaustive search: TSP, knapsack
  - Simple sorts: selection sort, bubble sort
  - Computing n!

Brute Force contd.

- Brute Force Algorithms refers to a programming style that does not include any shortcuts to improve performance, but instead relies on sheer computing power to try all possibilities until the solution to a problem is found.

Traveling salesman problem (TSP)

- Suppose a salesman needs to visit 10 cities across the country. How does one determine the order in which cities should be visited such that the total distance traveled is minimized? The brute force solution is simply to calculate the total distance for every possible route and then select the shortest one. This is not particularly efficient because it is possible to eliminate many possible routes through clever algorithms.

2. Greedy Algorithms

"take what you can get now" strategy

Work in phases.

In each phase the currently best decision is made

Greedy algorithms

- An optimization problem is one in which you want to find, not just *a* solution, but the *best* solution
- A "greedy algorithm" sometimes works well for optimization problems
- A greedy algorithm works in phases: At each phase:
  - You take the best you can get right now, without regard for future consequences
  - You hope that by choosing a *local* optimum at each step, you will end up at a *global* optimum

Greedy Algorithms - Examples

- **Dijkstra's algorithm**
  (shortest path is weighted graphs)
- **Prim's algorithm, Kruskal's algorithm**
  (minimal spanning tree in weighted graphs)
- **Coin exchange problem**
- **Huffman Trees**

Example: Counting money

- Suppose you want to count out a certain amount of money, using the fewest possible bills and coins
- A greedy algorithm would do this would be:
  At each step, take the largest possible bill or coin that does not overshoot
  - Example: To make K.shs 639/=, you can choose:
    - a 500 bill
    - a 100 shilling bill, to make 600/=
    - a 20 shilling coin, to make 620/=
    - a 10 shilling coin, to make 630/=
    - A 5 shilling coin, to make 635/=
    - four 1 shilling coins, to make 639/=

A failure of the greedy algorithm

- In some (fictional) monetary system, “krons” come in 1 kron, 7 kron, and 10 kron coins
- Using a greedy algorithm to count out 15 krons, you would get
  - A 10 kron piece
  - Five 1 kron pieces, for a total of 15 krons
  - This requires six coins
- A better solution would be to use two 7 kron pieces and one 1 kron piece
  - This only requires three coins
- The greedy algorithm results in a solution, but not in an optimal solution

3. Divide and Conquer

- **Reduce the problem to smaller problems (by a factor of at least 2) solved recursively and then combine the solutions**

Examples: Binary Search
Mergesort
Quick sort
Tree traversal

In general, problems that can be defined recursively

Divide and Conquer

- A **divide and conquer algorithm** consists of two parts:
  - Divide the problem into smaller subproblems of the same type, and solve these subproblems recursively
  - Combine the solutions to the subproblems into a solution to the original problem
- Traditionally, an algorithm is only called “divide and conquer” if it contains at least two recursive calls

Decrease and Conquer

- **Reduce the problem to smaller problems solved recursively and then combine the solutions**

Examples of decrease-and-conquer algorithms:

Insertion sort

Topological sorting

Binary Tree traversals: inorder, preorder and postorder (recursion)

Computing the length of the longest path in a binary tree (recursion)

Computing Fibonacci numbers (recursion)

Reversing a queue (recursion)

4. Dynamic Programming

Bottom-Up Technique in which the smallest sub-instances are ***explicitly*** solved first and the results of these used to construct solutions to progressively larger sub-instances.

Example:
Fibonacci numbers computed by iteration.

Dynamic programming algorithms

- A dynamic programming algorithm remembers past results and uses them to find new results
- Dynamic programming is generally used for optimization problems
  - Multiple solutions exist, need to find the “best” one
  - Requires “optimal substructure” and “overlapping subproblems”
    - **Optimal substructure**: Optimal solution contains optimal solutions to subproblems
    - **Overlapping subproblems**: Solutions to subproblems can be stored and reused in a bottom-up fashion
- This differs from Divide and Conquer, where subproblems generally need not overlap

5. Transform and Conquer

The problem is modified to be more amenable to solution. In the second stage the problem is solved.

- **Problem simplification e.g. presorting**

Example: finding the two closest numbers in an array of numbers.

Brute force solution: $O(n^2)$

Transform and conquer with presorting: O(nlogn)

- **Change in the representation**

Example: AVL trees guarantee O(nlogn) search time

- **Problem reduction**

Example: least common multiple: lcm(m,n) = (m*n)/ gcd(m,n)

6. Backtracking

Generate-and-Test methods

Based on exhaustive search in multiple choice problems

Typically used with depth-first state space search problems.

Example: Puzzles

Backtracking – State Space Search

- initial state
- goal state(s)
- a set of intermediate states
- a set of operators that transform one state into another. Each operator has preconditions and postconditions.
- a cost function – evaluates the cost of the operations (optional)
- a utility function – evaluates how close is a given state to the goal state (optional)

Backtracking algorithms

- Backtracking algorithms are based on a depth-first recursive search
- A backtracking algorithm:
  - Tests to see if a solution has been found, and if so, returns it; otherwise
  - For each choice that can be made at this point,
    - Make that choice
    - Recur
    - If the recursion returns a solution, return it
  - If no choices remain, return failure

Example backtracking algorithm

- To color a map with no more than four colors:
  - color(Country n):
    - If all countries have been colored (n > number of countries) return success; otherwise,
    - For each color c of four colors,
      - If country n is not adjacent to a country that has been colored c
        - Color country n with color c
        - recursively color country n+1
        - If successful, return success
    - If loop exits, return failure

7. Genetic Algorithms

- Search for good solutions among possible solutions
- The best possible solution may be missed
- A solution is coded by a string , also called chromosome. The words string and chromosome are used interchangeably
- A strings fitness is a measure of how good a solution it codes. Fitness is calculated by a fitness function
- *Selection:* The procedure to choose parents
- *Crossover* is the procedure by which two chromosomes mate to create a new offspring chromosome
- *Mutation :* with a certain probability flip a bit in the offspring

Basic Genetic Algorithm

1. **Start:** Generate random population of *n* chromosomes (suitable solutions for the problem)
2. **Fitness:** Evaluate the fitness *f(x)* of each chromosome *x* in the population
3. **New population:** Create a new population by repeating following steps until the new population is complete
4. **Test:** If the end condition is satisfied, stop, and return the best solution in current population

New Population

- **Selection: Select two parent chromosomes from a population according to their fitness**
- **Crossover: With a crossover probability cross over the parents to form a new offspring (children).** If no crossover was performed, offspring is an exact copy of parents**.**
- **Mutation: With a mutation probability mutate new offspring at each locus (position in chromosome).**

Conclusion

How to choose the approach?

First, by understanding the problem, and second, by knowing various problems and how they are solved using different approaches.

Read:

https://cgi.csc.liv.ac.uk/~ped/teachadmin/algor/algor.html